

ESCUELA POLITÉCNICA SUPERIOR
INFORMÁTICA – CURSO 2023-24
PRÁCTICA 4: ESTRUCTURAS CONDICIONALES

PRERREQUISITOS

- Haber comprendido y asimilado el contenido esencial de la/s práctica/s anterior/es.
- LEER el contenido de este documento previo a los ejercicios.

OBJETIVOS

En esta práctica aprenderemos cómo programar el realizar o no un conjunto de instrucciones diferentes dependiendo de si se da o no una determinada condición. Lo haremos mediante el uso de operaciones condicionales y de estructuras condicionales selectivas.

Así pues, los objetivos de esta práctica son los siguientes:

- Aprender el concepto de condición.
- Aprender los operadores más habituales empleados en programación, y particularmente en C++, para elaborar condiciones.
- Aprender la estructura condicional más habitual en programación, la estructura “if”, así como sus variantes extendidas “if-else” e “if-else if-else”, las cuales permiten indicar qué instrucciones realizar por el programa dependiendo de qué condición se cumpla.

Al finalizar esta práctica, el alumno debe ser capaz de:

- Elaborar condiciones mediante los operadores básicos de tipo relacionales y lógicos.
- Emplear la estructura “if” y sus variantes para elaborar programas que incluya la realización o no de una o varias instrucciones dependiendo de una determinada condición elaborada.

CONCEPTO DE CONDICIÓN

El concepto de condición es algo que asimilamos desde muy temprana edad, así como que si se cumple una condición (o un conjunto de condiciones) implica una consecuencia o acontecimiento. Es probable que cuando pequeño sus padres o tutores les haya dicho algo como “Si te portas bien tendrás una recompensa” o “Si te portas mal tendrás un castigo”. En esas frases reconocemos la condición y la acción o evento consecuente que tendría lugar en caso de que la condición se cumpliera. Ejemplos hay infinitos y forman parte de nuestro día a día (“Si llueve, cogeré paraguas”, “Si suben los precios de productos básicos, empobreceré”, “Si bebo, no debo conducir”, etc.).

Las condiciones que hemos usado como ejemplos son las más básicas, pero algunas condiciones son duales, es decir, hacemos una acción u otra dependiendo de si una condición se cumple o no se cumple. Por ejemplo “En mis vacaciones, si los vuelos están baratos, viajaré al extranjero, pero si no, me iré al pueblo”. Otras, concatenan varias condiciones y una determinada acción asociada: “Si mi nota es superior a 9, tendré un sobresaliente, si no, pero es superior a 7, tendré un notable, si no, pero es superior a 5, tendré un aprobado, si no, tendré un suspenso...”.

En funciones matemáticas también consideramos condiciones (las funciones que denominamos “por partes”), y, como no podría esperarse menos, en programación también están presentes. Es obvio, puesto que hay infinitas situaciones en las que queremos realizar una determinada acción o no dependiendo de nuestros intereses o del problema que queramos abordar mediante un programa o aplicación.

En esta práctica, aprenderemos la forma más básica y flexible (e intuitiva, dicho sea de paso) para **elaborar condiciones y conjuntos de instrucciones a realizar en caso de cumplirse. Es necesario**, por tanto, **conocer nuevos operadores**, que no sean aritméticos, sino **que sirvan para realizar una operación de comparación o establecer una regla lógica**. Una vez aprendido esto,

podemos elaborar las condiciones que **en combinación con una “estructura condicional” nos permita indicar qué parte del programa ejecutar** si se cumple o no una o un conjunto de condiciones.

OPERADORES QUE PERMITEN CREAR CONDICIONES

Podemos crear condiciones mediante operadores específicos. El resultado de estas operaciones siempre devolverá 0 ó 1, indicando con 0 que la condición NO se cumple (es falsa) e indicando con 1 que la condición SÍ se cumple (es verdad).

Expresiones relacionales

Son aquellas que permiten establecer condiciones de relación entre números. Las englobamos en la siguiente tabla:

Operador	Expresión Relacional	Resultado
>	val1 > val2	Es 1 (verdadero) si val1 es mayor que val2. Si no, el resultado es 0 (falso)
<	val1 < val2	Es 1 si val1 es menor que val2. Si no, es 0
>=	val1 >= val2	Es 1 si val1 es mayor o igual que val2. Si no, es 0
<=	val1 <= val2	Es 1 si val1 es menor o igual que val2. Si no, es 0
==	val1 == val2	Es 1 si val1 es igual a val2. Si no, es 0
!=	val1 != val2	Es 1 si val1 es distinto de val2. Si no, es 0

Donde val1 y val2 serían los datos numéricos con los que se realiza la operación, es decir, **los operandos**.

Expresiones lógicas

Estas expresiones tienen como operandos (los datos con los que se operan) variables o expresiones de tipo lógico, es decir, sólo deben tener valores 0 (falso) o 1 (verdadero). En la siguiente tabla indicamos los operadores lógicos más básicos:

Operador	Expresión lógica	Resultado
&& (Y, AND)	expr1 && expr2	Es 1 (verdadero) si tanto expr1 como expr2 son 1 (verdadero) . Si no (expr1 es 0 (falso), o lo es expr2, o lo son los dos), es 0 (falso) .
 (O, OR)	expr1 expr2	Es 0 (falso) si tanto expr1 como expr2 son 0 (falso) . Si no (expr1 es 1 (falso), o lo es expr2, o lo son los dos), es 1(verdadero).
! (Negación, NOT)	!expr	Es 1 (verdadero) si expr1 es 0 (falso), y es 0 (falso) si expr2 es 1 (verdadero) . Es un operador unario , es decir, trabaja sobre un único operando.

Donde expr1, expr2 y expr serían valores lógicos u otras expresiones lógicas o relacionales.

Orden de precedencia

Al igual que ocurre con las operaciones aritméticas, donde por ejemplo las divisiones y productos tienen preferencia en el cálculo sobre las sumas y las restas (si no hay paréntesis, las divisiones y multiplicaciones se realizan antes que las sumas y restas), en C++ las expresiones lógicas y relacionales también tienen su precedencia. Concretamente:

Mayor				Menor
<, <=, > y >=	== y !=	&&		

Recomendamos, en caso de duda, usar paréntesis para forzar el cálculo preferente de las operaciones que se requieran realizar.

EL TIPO BOOL PARA GUARDAR RESULTADOS

El tipo bool, que ya se mencionó en prácticas anteriores, es el tipo ideal para almacenar resultados de operaciones condicionales o relacionales.

Por ejemplo:

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int dato;
    bool es_menor_que_10; // variable tipo bool
    cout << "Introduzca un entero:";
    cin >> dato;
    // almacenamos el resultado de la comparación:
    es_menor_que_10 = (dato < 10);
    cout << "El número es menor que diez: " << es_menor_que_10;
    return 0;
}
```

ESTRUCTURA CONDICIONAL SELECTIVA: IF - ELSE IF – ELSE

Como ya se ha indicado, las condiciones se establecen principalmente para realizar una determinada acción, operación o conjunto de instrucciones si la condición se cumple (es verdadera), pero también se puede extender y distinguir entre la realización de un conjunto de instrucciones u otras dependiendo de si se cumple (verdadero) o NO se cumple (falso) una determinada condición. Yendo más allá, podemos relacionar varias condiciones entre sí y asociar a cada una la realización de un conjunto de acciones distintas.

Primera opción: uso de sólo if

La estructura más sencilla es la estructura if a secas. Esta permite indicar que en caso de que se cumpla una condición, se ejecute un conjunto de instrucciones de código. La sintaxis para hacer uso de esta estructura en C++ es la siguiente:

```
if (condicion) {
    sentencia1; // Se realiza lo que haya entre llaves...
    sentencia2; // SI Y SOLO SI se cumple la condición
    ...
}
```

Donde **condicion** debe ser un valor lógico (0 o 1) o un conjunto de expresiones lógicas y/o relacionales que finalmente darán o 0 o 1 al realizar los cálculos.

Un ejemplo:

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int dato;
    cout << "Introduzca un entero:";
    cin >> dato;
    if(dato < 0){
        cout << "El número es negativo";
    }
}
```

```

        cout << endl;
    }
    return 0;
}

```

Este programa pide un número de tipo entero al usuario y mediante una condición (`dato < 0`) y la estructura `if(...){...}` podemos comprobar si el número es negativo, en cuyo caso, hemos indicado que la acción que ha de realizar el programa es indicar por consola (pantalla) que el número es negativo.

Preste especial atención a los detalles de la sintaxis: **en primer lugar**, viene la palabra clave “**if**”, **seguido de paréntesis (...)** que **abarcan la condición** y seguido de **llaves {...}** que **engloban todo el código que se quiere realizar**.

Segunda opción: dos bifurcaciones con el uso de if-else

A menudo nos interesa realizar otro conjunto de acciones distintas en caso de que NO se haya cumplido una determinada condición. Para ello, podemos extender la estructura `if` a su forma `if-else`. La estructura Tiene esta sintaxis:

```

if (condicion) {
    sentencia1; // Se realiza lo que haya entre llaves...
    sentencia2; // SI Y SOLO SI se cumple la condición
    ...
} else {
    Sentencia2; // Se realiza lo que haya entre llaves...
    Sentencia3; // si NO se cumple la condición
    ...
}

```

Podemos apreciar que **la primera parte de esta estructura no cambia** con respecto a lo que hemos visto en la estructura sencilla `if`. Añadiendo **justo a continuación** `else{...}` podemos establecer qué hacer en caso de que la condición NO se cumpla (es decir, resultase falsa, 0).

De aquí debemos tener claro lo siguiente:

- **Poner la parte “else{...}” sin tener un if justo antes previamente, es un absurdo** y dará un error de compilación. Esto es muy lógico, imagínese que alguien empieza una conversación así: “si no, tendré que avisar...”, “Si no” ¿qué? No tendría sentido, es necesaria una condición previa sobre la que apoyarse para entender el “si no”, es decir, el “else” o “en caso contrario”.
- **Poner una condición adicional acompañando a else (algo como “else(condicion2){...}” tampoco tendría sentido.** “Else” significa literalmente en español: “En caso contrario”, **la condición siempre será lo contrario a la condición establecida en el if**, por lo tanto, no es necesario que acompañe una condición al else, y de hacerlo dará nuevamente un error de compilación.

Otro ejemplo

```

#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int dato;
    cout << “Introduzca un entero:”;
    cin >> dato;
}

```

```

if(dato < 0){
    cout << "El número es negativo";
} else{
    cout << "El número es positivo o cero";
}
return 0;
}

```

En esta ocasión, extendemos el programa anterior para que en caso de que la condición no sea cierta, indique por consola que el número introducido es positivo o bien es 0.

Tercera opción: abordar múltiples condiciones con el uso de if - else if -else

Si nos fijamos en el ejemplo anterior, quizás le surja la pregunta: ¿Es posible mostrar un tercer mensaje alternativo si el número introducido es 0? Pues SÍ, es posible. Y aunque no es necesario hacerlo como se va a indicar, vamos a aprovechar el dar **una solución** usando la tercera forma de la estructura selectiva: **if – else if – else**.

Básicamente, podemos considerar varias condiciones $\{c_1, c_2, \dots, c_{n-1}, c_n\}$ para considerar distinto código a ejecutar, pero relacionadas de forma **que si las anteriores $c_1, \dots, c_{i-1}$ NO se han cumplido, PERO SÍ se cumple la condición c_i , es el código asociado a esta condición el que se ejecuta**.

La estructura es simple:

```

if (condición_1){
    sentencia1; // Se realiza lo que haya entre llaves...
    sentencia2; // SI Y SOLO SI se cumple la condición
    ...
} else if(condición_2){
} else if(condición_3){
}.... // Pueden haber varias condiciones
else{ // Este else final es opcional
    Sentencia2; // Se realiza lo que haya entre llaves...
    Sentencia3; // si NO se cumplen las anteriores
    ... // condiciones
}

```

Así pues, para considerar con esta estructura un tercer mensaje alternativo, sería:

```

#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int dato;
    cout << "Introduzca un entero:";
    cin >> dato;
    if(dato < 0){
        cout << "El número es negativo";
    }
}

```

```

    } else if (dato == 0){
        cout << "El número es cero";
    } else{
        cout << "El número es positivo";
    }
    return 0;
}

```

Y con esto tendríamos un programa válido para considerar el mostrar tres mensajes diferentes dependiendo de la naturaleza del número entero introducido.

Para reflexionar: por las características del problema que tratábamos de solucionar (mostrar si el número era positivo, negativo o cero), podríamos haber hecho un programa también sencillo con tres if, en cada uno las condiciones “dato < 0”, “dato > 0” y “dato == 0”. Estos if podrían ordenarse de la manera que fuera en el programa, dado que un número entero sólo puede cumplir una y sólo una de las tres condiciones consideradas. Pero, de cara al comportamiento del código, cada if sería independiente el uno del otro.

Para acabar, algunos aspectos a tener en cuenta:

- “Else if” tiene la particularidad de que la ejecución del código que abarca está condicionada a que las anteriores condiciones (“if” y “else if” anteriores) hayan sido falsas.
- Es válida la estructura if – else if, es decir, sin un “else” final. Esto permite que no tengamos que considerar más casos de los necesarios (quizás no nos interese hacer algo en particular en caso de que no se haya cumplido ninguna de las condiciones anteriores)
- Si se usa, “else” debe estar siempre al final de la estructura condicional, ya que contemplar el caso en el que NINGUNA de las condiciones anteriores consideradas se hayan cumplido.
- Como cabría esperar, se pueden crear varias estructuras if y sus variantes a lo largo del programa. No se está limitado al uso de una única.
- También como cabría imaginar, podemos seguir creando código adicional después de crear una estructura de este tipo.
- Si declaramos variables dentro del cuerpo (lo que va entre llaves) del if, else if o else, esa variable sólo podrá usarse en ese cuerpo (lo que denominamos el contexto, que lo delimitan las llaves que abarca el código).
- Podemos hacer uso de estas estructuras en funciones, no sólo en la función principal.

EJERCICIOS

1. Cree un programa con el siguiente código, que pretende devolver 0 si se introduce un número negativo y el valor introducido si el número es mayor o igual que cero:

```

#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int dato;
    cout << "Introduzca un entero:";
    cin >> dato;
    if(dato < 0){
        int res = 0;
    } else{

```

```

        res = dato;

    }

    cout << res;

    return 0;

}

```

Compruebe que al ejecutar el código **hay errores de compilación**.

¿Qué problema cree que puede haber? Ayúdese de las indicaciones que da Zinjai sobre el error y **arregle el programa modificando sólo dos líneas de código**. Consulte a su profesor si no da con la razón del problema o no lo comprende.

2. Cree un nuevo programa que pida por teclado dos valores enteros (a y b) y compruebe si el primero (a) es múltiplo del segundo (b). En caso afirmativo, el programa debe imprimir el mensaje: “El número a es múltiplo de b ” y en caso contrario “El número a NO es múltiplo de b ”.

Nota: Para saber si un número es múltiplo de otro, use el operador módulo % que calcula el resto de la división. Por ejemplo, $a \% b$ da como resultado el resto de la división entera del valor de la variable a dividida entre b . Un número es múltiplo de otro si la división entera es exacta, es decir, el resto es cero.

3. Escriba un programa que lea un carácter por teclado y realice acciones diferentes según el caso:
 - b) Si es una letra mayúscula, mostrará la misma letra en minúscula.
 - a) Si es una letra minúscula, mostrará la misma letra en mayúscula.
 - b) Si es un dígito, mostrará un mensaje que lo indique.
 - c) Para otro carácter, mostrará el código ASCII del carácter introducido.
4. Elabore un nuevo programa que resuelva una ecuación de segundo grado ($ax^2 + bx + c = 0$), pidiendo por pantalla los datos necesarios y mostrando las dos soluciones. Recuerde que las soluciones de una ecuación de segundo grado son:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Para realizar la raíz cuadrada necesita usar la función `sqrt()` incluida en la librería `cmath`.

Modifique el programa para tener en cuenta, además del caso básico, los casos en los que la ecuación de segundo grado tiene una única solución o no tiene soluciones reales, comprobándolo e indicándolo en los resultados mostrados al usuario.

Nota: Recuerde que una ecuación de segundo grado tiene:

- Dos soluciones reales si: $b^2 - 4ac > 0$.
- Una solución real si: $b^2 - 4ac = 0$.
- Ninguna solución real si: $b^2 - 4ac < 0$.

Ejercicios con implementación de funciones

5. Comience un nuevo programa e implemente lo siguiente:
 - 1) Cree una función con el siguiente prototipo:

`float f(float x);`

Defina la función para que implemente la siguiente función matemática por partes:

$$f(x) = \begin{cases} -1 & x \leq 1 \\ \log x, & 1 < x \leq 2 \\ \frac{1}{x-2} & x > 2 \end{cases}$$

Nota: use la librería `cmath` para calcular el logaritmo en base 10, que se requiere para hacer la implementación. Infórmese de cuál es la función apropiada buscando en la web.

- 2) En la función principal, programe la solicitud de un dato por consola y muestre por consola el resultado. Con el programa ya creado, pruebe a introducir distintos valores y compruebe que funciona correctamente.

6. Comience un nuevo programa y realice lo siguiente:

- 1) Cree una función con el siguiente prototipo:

`char convierte_caracter(char c);`

Defina la función para que a partir del dato introducido como parámetro `c`, devuelva un carácter diferente siguiendo las mismas condiciones que en el ejercicio 3 de este boletín (tanto si es un dígito como otro carácter que no sea una letra, la función devolverá ese mismo carácter).

- 2) Con la función ya definida, programe en la función principal la solicitud de un carácter por consola, llame a la función creada en el apartado 1 para calcular el carácter resultante y muéstrela por pantalla.

7. Comience un nuevo programa y realice lo siguiente:

- 1) Elabore una función con el siguiente prototipo:

`float calificacion_final(float nota_teoría, float nota_práctica);`

- 2) Defina la función para que calcule la nota final (sobre 10 puntos). Esta nota se divide en un 60% de nota de prácticas y un 40% de nota de teoría.
- 3) En la función principal, realice lo siguiente:
 1. Solicite al usuario un valor de nota de teoría y un valor de nota de práctica.
 2. Programe la comprobación de que cada nota esté comprendida en un rango de valores correcto, es decir, entre 0 y 10. Si alguno de los valores no es correcto, debe mostrar por pantalla el mensaje “Error: alguna de las notas tiene un valor incorrecto”. Si los datos son correctos, use la función implementada para calcular la nota final y muéstrela por pantalla.
 3. Complete el programa para que de forma complementaria a la calificación obtenida, muestre por pantalla el mensaje “SUSPENSO”, “APROBADO”, “NOTABLE” o “SOBRESALIENTE” según la nota final obtenida.

8. Realice un nuevo programa y elabore lo siguiente (los puntos 1,2 y 3 son comunes un ejercicio de la práctica 3):

- 1) Cree una función `resistencias_serie`, que reciba dos parámetros a modo de valores de resistencia. La función ha de calcular el valor de resistencia equivalente para dos resistencias dispuestas en serie en un circuito eléctrico.

Nota: El valor de la resistencia equivalente a las resistencias conectadas en **serie** es igual a la suma de los valores de cada una de ellas.

- 2) Cree una función `resistencias_paralelo`, que reciba dos parámetros a modo de valores de resistencia. La función ha de calcular el valor de resistencia equivalente para dos resistencias dispuestas en paralelo en un circuito eléctrico.

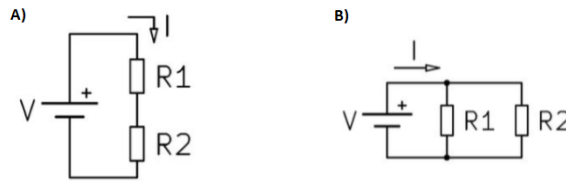
Nota: La resistencia total equivalente de un circuito de resistencias en **paralelo** es igual al recíproco de la suma de los inversos de las resistencias individuales. Se calcula según la siguiente ecuación: $1 / (1/R1 + 1/R2)$.

- 3) Cree una función `calcula_intensidad`, que reciba como parámetros un valor de resistencia y un valor de voltaje para una batería. La función debe devolver el valor de intensidad que circula por un circuito que contiene esos dos elementos.

Nota: Para calcular la intensidad que circula por los circuitos con la resistencia equivalente, tenga en cuenta la ley de Ohm: cuando por una resistencia eléctrica “R”,

circula una corriente "I", se produce en ella una caída de tensión "V" entre los extremos de la resistencia cuyo valor viene dado por: $V = I * R$.

- 4) Programe ahora la función principal. En ella, **solicite al usuario** el valor de dos resistencias y un valor de voltaje. Además, pida al usuario que elija la configuración de las resistencias en un circuito, siendo una de ellas dispuestas en serie (mostrado en la figura como circuito A) y otra dispuestas en paralelo (como el circuito B). En base a la decisión, calcule mediante el uso de las funciones elaboradas posteriormente el valor de la resistencia equivalente y la intensidad del circuito elegido, y muéstrelo por consola.



9. Realice un nuevo programa y elabore lo siguiente:

- 1) Cree las funciones que calculan el área de un triángulo, el área de un círculo y el área de un cuadrado. **Nota:** recuerde también crear una constante PI.
- 2) Cree una función que calcule el volumen de una figura geométrica de tipo prisma o cilindro. Esta función recibe dos parámetros: área de la base y altura, de modo que el volumen lo calcula multiplicando estas dos propiedades. El prototipo sería el siguiente:

`float area_prisma(float area_base, float altura);`

- 1) Cree una función que calcule el volumen de una figura geométrica de tipo pirámide o cono. Esta función recibe dos parámetros: área de la base y altura, de modo que el volumen lo calcula multiplicando estas dos propiedades y dividiendo entre 3. El prototipo sería el siguiente:

`float area_piramide(float area_base, float altura);`

- 2) En la función principal, solicite al usuario los datos que considere necesario para indicar el volumen de qué figura geométrica quiere calcular (de tipo prisma o pirámide, y tipo de base). En base a ello, solicite al usuario los datos necesarios para el cálculo, llame a las funciones implementadas adecuadas para cada caso y muestre un mensaje por pantalla coherente junto al resultado.

10. Se pide realizar un programa que realice varios cálculos de interés. Para ello, se va a seguir la siguiente secuencia de pasos:

- a) Inicialmente, la función principal (*main*), ha de mostrar por pantalla un menú de opciones que indique los cálculos que se pueden realizar:

-----Seleccione la operación a realizar-----

- 1: Calcular resistencia equivalente serie ($R1 + R2$)
- 2: Calcular resistencia equivalente paralelo ($R1 || R2$)
- 3: Calcular calor disipado por una resistencia
- 4: Calcular autonomía de un SAI

- b) Solicite por teclado un número entre 1 y 4, que indicará el cálculo que quiere realizar el usuario.

- c) Organice una estructura de selección múltiple *if /else if/ else* para implementar las operaciones asociadas a cada una de las posibles opciones que puede haber introducido el usuario.

- d) En el bloque correspondiente a cada opción, habrá que solicitar los datos necesarios y almacenarlos en variables para, posteriormente, realizar el cálculo correspondiente y mostrar

el resultado por pantalla.

Nota: Cada una de las operaciones que se piden elaborar deben estar implementadas como funciones. Para las operaciones 1 y 2, las orientaciones sobre cómo abordarlos los tiene en el ejercicio 8 de esta práctica; para las operaciones 3 y 4, las ecuaciones a usar se encuentran en la práctica 2, en los ejercicios 5 y 6 respectivamente.

ATC-US